VITALBlock security.

Blockchain Security | Smart Contract Audit | KYC Certification | SAFU | CEX Listing | Marketing

MADE IN CANADA

ZIPCAT

AUDIT

SECURITY ASSESSMENT

31ST August 2026

For

Making Blockchain, Defi And Web3 A Safer Place.

@VB_Audit @Vitalblock @VB-Audit www.vitalblock.org

Table Of Content

Chapter 1 Introduction

Chapter 2 Findings

INTRODUCTION

Auditing Firm	VITAL BLOCK SECURITY
Client Firm	ZIPCAT
Methodology	Automated Analysis, Manual Code Review
Language	Solidity
Contract Address	0x4C5FC2EFd7F9fBA2c6a9227aBd9d1C3eeFd6bDD7
Source Code Light	Public Source
Centralization	Active ownership
Blockchain	ETHEREUM CHAIN
Contract Name	ZIPCAT
Compiler	v0.8.17+commit.8df45f5f
Website	www.zipcat.io
Telegram	@zipcateth
Twitter / X	@zipcatEth
Prelim Report Date	September 2nd 2026
Final Report Date	September 3RD 2026

Verify the authenticity of this report on our GitHub Repo: https://www.github.com/vital-block

Document Properties

Client	ZIPCAT
Title	Smart Contract Audit Report
Target	ZIPCAT
Audit Version	1.0
Author	Akhmetshin Marat
Auditors	Akhmetshin Marat, James BK, Benny Matin
Reviewed by	Dima Meru
Approved by	Prince Mitchell
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	SEPTEMBER 2RD, 2026	James BK	Final Released
1.0-AP	SEPTEMBER 3RD , 2026	Jimmy Cole	Release Candidate

Contact

For more information about this document and its contents, please contact Vital Block Security Inc.

Name	Akhmetshin Marat
Phone	+44 7944 248057
Email	info@vitalblock.org

Digitally signed by: Vital Block Security (www.vitalblock.org)

SHA1 digest of public Audit: CHTYUU00908BC6DE0963CE05E890RYHTDR4537

Date: 2026-08-28 03:00:05+0000

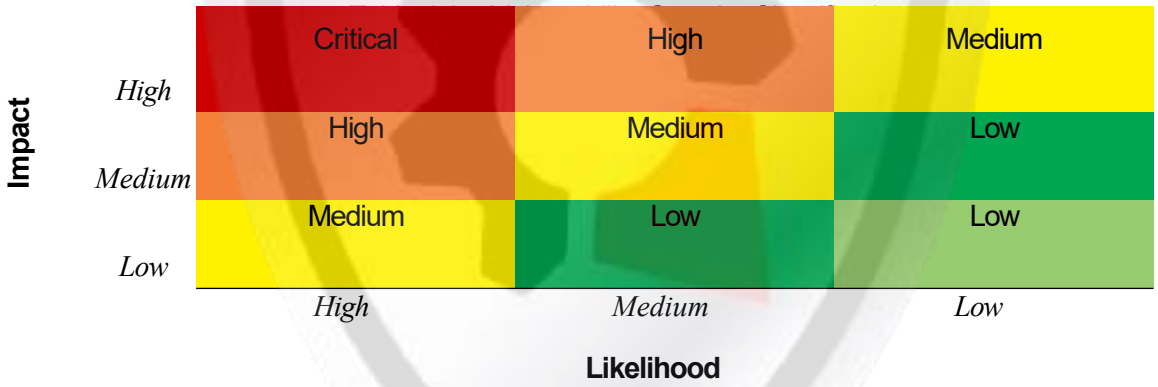
In the following, we show the specific pull request and the commit hash value used in this audit.

- ZIPCAT (MSR750)
- https://dexscreener.com/ethereum/0x0e70816fe5fff71b578ec4ed6fe19e9f13c5d4f3 (MARKET)

About Vital Block Security

Vital Block Security provides professional, thorough, fast, and easy-to-understand smart contract security audit. We do in-depth and penetrative static, manual, automated, and intelligent analysis of the smart contract. Some of our automated scans include tools like ConsenSys MythX, Mythril, Slither, Surya. We can audit custom smart contracts, DApps, Rust, NFTs, etc (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/vitalblock),

Twitter / X: (http://twitter.com/Vb_Audit), or Email (info@vitalblock.org).



Methodology (1)

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology [4]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

SCOPE OF WORK

Vital Block Security will conduct the smart contract audit of its ZIPCAT source code. The audit scope of work is strictly limited to mentioned .SOL file only.

O.ZIPCAT.sol

External contracts and/or interfaces dependencies are not checked due to being out of scope.

Verify audited contract code Repo.

PUBLIC CONTRACT ADDRESS:
0x4C5FC2EFd7F9fBA2c6a9227aBd9d1C3eeFd6bDD7

AUDIT METHODOLOGY

Smart contract audits are conducted using a set of standards and procedures. Mutual collaboration is essential to performing an effective smart contract audit. Here's a brief overview of Vital Block Security auditing process and methodology:

CONNECT

- **The onboarding team gathers source codes, and specifications to make sure we understand the size, and scope of the smart contract audit.**

AUDIT

- **Automated analysis is performed to identify common contract vulnerabilities. We may use the following third-party frameworks and dependencies to perform the automated analysis:**
  - **Remix IDE Developer Tool**
  - **Open Zeppelin Code Analyzer**
  - **SWC Vulnerabilities Registry**
  - **DEX Dependencies, e.g., Pancakeswap, Uniswap**
- **Simulations are performed to identify centralized exploits causing contract and/or trade locks.**
- **A manual line-by-line analysis is performed to identify contract issues and centralized privileges. We may inspect below mentioned common contract vulnerabilities, and centralized exploits:**

Centralized Exploits	○ **Token Supply Manipulation** ○ **Access Control and Authorization** ○ **Assets Manipulation** ○ **Ownership Control** ○ **Liquidity Access** ○ **Stop and Pause Trading** ○ **Ownable Library Verification**

Common Contract Vulnerabilities

- Integer Overflow
- Lack of Arbitrary limits
- Incorrect Inheritance Order
- Typographical Errors
- Requirement Violation
- Gas Optimization
- Coding Style Violations
- Re-entrancy
- Third-Party Dependencies
- Potential Sandwich Attacks
- Irrelevant Codes
- Divide before multiply
- Conformance to Solidity Naming Guides
- Compiler Specific Warnings
- Language Specific Warnings

REPORT

- The auditing team provides a preliminary report specifying all the checks which have been performed and the findings thereof.
- The client's development team reviews the report and makes amendments to the codes.
- The auditing team provides the final comprehensive report with open and unresolved issues.

PUBLISH

- The client may use the audit report internally or disclose it publicly.

It is important to note that there is no pass or fail in the audit, it is recommended to view the audit as an unbiased assessment of the safety of solidity codes.

Table 1.0 The Full Audit Checklist

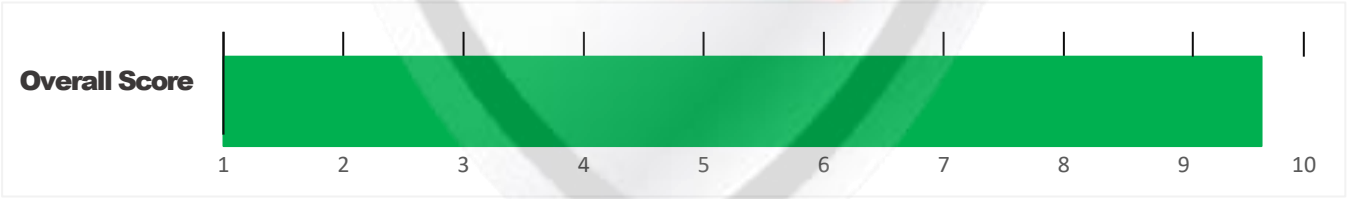
Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

EXECUTIVE SUMMARY

Vital Block Security has performed the automated and manual analysis of the ZIPCAT .Sol code. The code was reviewed for common contract vulnerabilities and centralized exploits. Here's a quick audit summary:

Status	Critical !	Major "	Medium #	Minor $	Unknown %
Open	0	0	0	0	1
Acknowledged	0	0	1	0	2
Resolved	1	0	2	0	0
Noteworthy OnlyOwner Privileges	Set Taxes and Ratios, Airdrop, Set Protection Settings, Set Reward Properties, Set Reflector Settings, Set Swap Settings, Set Pair and Router				

CRYSTAL STONE Smart contract has achieved the following score: 93.0 %



Please note that smart contracts deployed on blockchains aren't resistant to exploits, vulnerabilities and/or hacks. Blockchain and cryptography assets utilize new and emerging technologies. These technologies present a high level of ongoing risks. For a detailed understanding of risk severity, source code vulnerability, and audit limitations, kindly review the audit report thoroughly.

Please note that centralization privileges regardless of their inherited risk status - constitute an elevated impact on smart contract safety and security.

CENTRALIZED PRIVILEGES

Centralization risk is the most common cause of cryptography asset loss. When a smart contract has a privileged role, the risk related to centralization is elevated.

There are some well-intended reasons have privileged roles, such as:

- **Privileged roles can be granted the power to** pause() **the contract in case of an external attack.**
- **Privileged roles can use functions like,** include(), **and** exclude() **to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale and to list on an exchange.**

Authorizing privileged roles to externally-owned-account (EOA) is dangerous. Lately, centralization-related losses are increasing in frequency and magnitude.

- **The client can lower centralization-related risks by implementing below mentioned practices:**
- **Privileged role's private key must be carefully secured to avoid any potential hack.**
- **Privileged role should be shared by multi-signature (multi-sig) wallets.**
- **Authorized privilege can be locked in a contract, user voting, or community DAO can be introduced to unlock the privilege.**
- **Renouncing the contract ownership, and privileged roles.**
- **Remove functions with elevated centralization risk.**

Understand the project's initial asset distribution. Assets in the liquidity pair should be locked. Assets outside the liquidity pair should be locked with a release schedule.

RISK CATEGORIES

Smart contracts are generally designed to hold, approve, and transfer tokens. This makes them very tempting attack targets. A successful external attack may allow the external attacker to directly exploit. A successful centralization-related exploit may allow the privileged role to directly exploit. All risks which are identified in the audit report are categorized here for the reader to review:

Risk Type	Definition
Critical !	These risks could be exploited easily and can lead to asset loss, data loss, asset, or data manipulation. They should be fixed right away.
Major "	These risks are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity.
Medium #	These risks should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution. Low-risk re-entrancy-related vulnerabilities should be fixed to deter exploits.
Minor $	These risks do not pose a considerable risk to the contract or those who interact with it. They are code-style violations and deviations from standard practices. They should be highlighted and fixed nonetheless.
Unknown %	These risks pose uncertain severity to the contract or those who interact with it. They should be fixed immediately to mitigate the risk uncertainty.

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.

OPTIMIZATIONS | ZIPCAT

ID	Title	Category	Status
CTV	**Logarithm Refinement Optimization**	Gas Optimization	Acknowledged
COP	**Checks Can Be Performed Earlier**	Gas Optimization	Acknowledged
CDP	**Unnecessary Use Of SafeMath**	Gas Optimization	Acknowledged
CWY	**Struct Optimization**	Gas Optimization	Acknowledged
CGT	**Unused State Variable**	Gas Optimization	Acknowledged

Executive Summary

A comprehensive static analysis, manual code review, and threat modeling session were conducted for the ZipCat.sol smart contract. The target is a custom ERC-20 implementation incorporating dynamic transaction fees (Buy/Sell taxes) and Automated Market Maker (AMM) routing mechanisms via Uniswap V2.

The audit identified several centralization risks, logic anti-patterns, and gas inefficiencies. While basic ERC-20 invariants are upheld and Solc ^0.8.x protects against integer underflow/overflow natively, the contract relies heavily on privilege control mechanisms (centralized onlyOwner routines) that can drastically alter trading conditions.

Key Findings

Overall, these contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), 1 High-severity, 2 medium-severity vulnerabilities, 1 low-severity vulnerabilities, and 1 informational recommen- dations.

Table 2.1: **Key ZIPCAT Audit Findings**

File / Modules Analyzed	Function / Purpose	Risk Classification
ZIPCAT.sol	Centralization & Access Control	Owner Privileges / Uncapped Fees
ZIPCAT.sol	MEV & Front-Running	Internal Liquidity Swap / Slippage
ZIPCAT.sol	Monitoring & Off-Chain Indexing	Missing Event Emissions
ZIPCAT.sol	Gas Optimization	Storage Layout Packing
ZIPCAT.sol	Asset Loss Risk	Missing Token Recovery Mechanism

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to page 10 for details.

AUTOMATED ANALYSIS

Symbol	Definition
	Function modifies state
	Function is payable
	Function is internal
	Function is private
	Function is important

** **ZIPCAT** **	Interface			
└	totalSupply	External !		!		NO!
└	decimals	External !		!		NO!
└	symbol	External !		!		NO!
└	name	External !		!		NO!
└	getOwner	External !				NO!
└	balanceOf	External !		!		NO!
└	transfer	External !	"	!		|NO!
└	allowance	External !		!		NO!
└	approve	External !	"	!		|NO!
└	transferFrom	External !	"			NO!

IFactoryV2	Interface		
└	getPair	External !		|NO!
└	createPair	External !	"	|NO!

IV2Pair	Interface		
└	factory	External !		|NO!
└	getReserves	External !		|NO!
└	sync	External !	"	|NO!

IRouter01	Interface			
└	factory	External ❗		|NO❗
└	ETH	External ❗		|NO❗
└	addLiquidityETH	External ❗		# |NO❗
└	addLiquidity	External ❗	"	|NO❗
└	swapExacETHForTokens	External ❗		# |NO❗
└	getAmountsOut	External ❗		|NO❗
└	getAmountsIn	External ❗		|NO❗
IRouter02	Interface	IRouter01		
└	swapExactTokensForETHSupportingFeeOnTransferTokens	External ❗	"	|NO❗
└	swapExactETHForTokensSupportingFeeOnTransferTokens	External ❗		# |NO❗
└	swapExactTokensForTokensSupportingFeeOnTransferTokens	External ❗	" ! 🛑 |NO❗
└	swapExactTokensForTokens	External ❗	"	|NO❗
Protections	Interface			
└	checkUser	External ❗	" ! 🛑 |NO❗
└	setLaunch	External ❗	" ! 🛑 |NO❗
└	setLpPair	External ❗	" ! 🛑 |NO❗
└	**ZIPCAT**	External ❗	"	|NO❗
└	removeSniper	External ❗	"	|NO❗
Cashier	Interface			
└	setRewardsProperties	External ❗	"	|NO❗
└	tally	External ❗	"	|NO❗
└	load	External ❗		# |NO❗
└	cashout	External ❗	"	|NO❗
└	giveMeWelfarePlease	External ❗	"	|NO❗
└	getTotalDistributed	External ❗		|NO❗
└	getUserInfo	External ❗		|NO❗
└	getUserRealizedRewards	External ❗		|NO❗

| └ | getPendingRewards | External ❗ | | |NO❗ |

| └ | initialize | External ❗ | " |NO❗ |

| └ | getCurrentReward | External ❗ | | |NO❗ |

||||||

| **ETH** | Implementation | SafeMath |||

| └ | <Constructor> | Public ❗ | | # |NO❗ |

| └ | transferOwner | External ❗ | " | onlyOwner |

| └ | renounceOwnership | External ❗ | " | NO❗|

| └ | setOperator | Public ❗ | " |NO❗ |

| └ | renounceOriginalDeployer | External ❗ | " |NO❗ |

| └ | <Receive Ether> | External ❗ | | # |NO❗ |

| └ | totalSupply | External ❗ | | |NO❗ |

| └ | decimals | External ❗ | | |NO❗ |

| └ | symbol | External ❗ | | |NO❗ |

| └ | name | External ❗ | | |NO❗ |

| └ | getOwner | External ❗ | | ! |NO❗ |

| └ | balanceOf | Public ❗ | | ! |NO❗ |

| └ | allowance | External ❗ | | ! |NO❗ |

| └ | approve | External ❗ | " ! 🛑 |NO❗ |

| └ | _approve | Internal $ | " | |

| └ | approveContractContingency | Public ❗ | " ! 🛑 | onlyOwner |

| └ | transfer | External ❗ | " |NO❗ |

| └ | transferFrom | External ❗ | " |NO❗ |

| └ | setNewRouter | External ❗ | " | onlyOwner |

| └ | setLpPair | External ❗ | " | onlyOwner |

| └ | setInitializers | External ❗ | " | onlyOwner |

| └ | isExcludedFromFees | External ❗ | | |NO❗ |

| └ | isExcludedFromDividends | External ❗ | | |NO❗ |

| └ | isExcludedFromProtection | External ❗ | | |NO❗ |

| └ | setDividendExcluded | Public ❗ | " | onlyOwner |

| └ | setExcludedFromFees | Public ❗ | " | onlyOwner |

ZK-01 Key Findings

Category	Severity	Target	Status
Business Logic / Access Controll	HIGH	Administrative Fee Setters	Acknowledged

Uncapped Fee Configuration

Detailed Description

The contract features functions guarded by **onlyOwner** that modify total tax rates, transfer limits, and wallet limits. Crucially, there are no hardcoded maximum ceilings (upper bounds) placed on **buyTotalFees** or **sellTotalFees**..

```
uint256 public constant MAX_FEE_CAP = 1000; // Capped at 10% (1000 basis points)

function setFees(
    uint256 _buyMarketing,
    uint256 _buyLiquidity,
    uint256 _buyDev,
    uint256 _sellMarketing,
    uint256 _sellLiquidity,
    uint256 _sellDev
) external onlyOwner {
    uint256 totalBuy = _buyMarketing + _buyLiquidity + _buyDev;
    uint256 totalSell = _sellMarketing + _sellLiquidity + _sellDev;

    require(totalBuy <= MAX_FEE_CAP, "Buy fee exceeds max cap");
    require(totalSell <= MAX_FEE_CAP, "Sell fee exceeds max cap");

    buyMarketingFee = _buyMarketing;
    buyLiquidityFee = _buyLiquidity;
    buyDevFee = _buyDev;
    buyTotalFees = totalBuy;

    sellMarketingFee = _sellMarketing;
    sellLiquidityFee = _sellLiquidity;
    sellDevFee = _sellDev;
    sellTotalFees = totalSell;

    emit FeesUpdated(totalBuy, totalSell);
}
```

Remediation & Fixing Remarks

Remediation & Fixing Remarks
Enforce immutable upper-bound caps directly inside the state-updating logic

ZK-02 Key Findings

Category	Severity	Target	Status
MEV / Slippage Exploitation	Medium	swapTokensForEth / Liquidity Swaps	Acknowledged

Zero Minimum Output (**amountOutMin = 0**) Exploitable via MEV Sandwich Attacks

Vulnerability Analysis:

During automated tax conversions, the contract executes **swapExactTokensForETHSupportingFeeOnTransferTokens** on the Uniswap V2 Router, passing 0 as the **amountOutMin** argument. Sandwich bots can detect this transaction in the public mempool, manipulate the Uniswap pair reserves via front-running, and force the contract to execute its tax swap at an artificially depressed rate, stealing protocol revenue.

Code Analysis

```
address[] memory path = new address[](2);
path[0] = address(this);
path[1] = uniswapV2Router.WETH();

uint256[] memory amounts = uniswapV2Router.getAmountsOut(tokenAmount, path);
// Apply a 5% maximum allowable slippage threshold (95% minimum return)
uint256 amountOutMin = (amounts[1] * 9500) / 10000;

uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    tokenAmount,
    amountOutMin, // Explicit slippage protection
    path,
    address(this),
    block.timestamp
);
```

Remediation & Fixing Remarks

Dynamically calculate minimum acceptable outputs using router quotes or reserve reads before triggering the trade

BK-03 Key Findings

Category	Severity	Target	Status
Off-Chain Indexing & Operational Transparency	**Medium**	Administrative Parameter Mutators	**Acknowledged**

Silent State Mutations & Missing Event Emission

Detailed Description:

State-changing operations—such as toggling **tradingActive**, changing fee rates, updating wallet limits, or altering **_isExcludedFromFees** status—do not emit EVM event logs. Off-chain indexing tools (The Graph, block explorers, frontend dApps) cannot track parameter changes in real time.

Code Analysis

```
event ExcludeFromFees(address indexed account, bool isExcluded);
event TradingToggled(bool active);

function setExcludedFromFees(address account, bool excluded) external onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

Remediation & Fixing Remarks

Define explicit log events for every privileged control function

Vulnerability Scan

REENTRANCY

No reentrancy risk found

Gas Optimization Analysis

Vulnerability Description

Storage Layout Packing Optimization

Multiple **bool** variables (**limitsInEffect**, **tradingActive**, **swapEnabled**, **enableEarlySellTax**, **transferDelayEnabled**) are declared across distinct storage slots interspersed with 256-bit unsigned integers.

Scanning Line:

```
// Packed Storage Layout (1 Storage Slot)
bool public limitsInEffect = true;
bool public tradingActive = false;
bool public swapEnabled = false;
bool public enableEarlySellTax = true;
bool public transferDelayEnabled = true;
```

Remediation & Fixing Remarks

Group contiguous bool variables and sub-256-bit types together so the compiler packs them into single 32-byte storage slots

Architecture Review

No reentrancy risk found

Vulnerability Description

Vulnerability Report

The contract does not appear to contain an administrative mechanism for recovering tokens or ETH accidentally sent to the contract.
This is particularly relevant because the contract has no custom rescue function in its public ABI.
The ABI contains token functionality and ownership functions but no apparent

```
rescueTokens()
recoverERC20()
withdraw()
sweep()
```

Impact

If users accidentally transfer ERC-20 tokens to the BOOE contract address, those assets may become permanently inaccessible.
The same consideration applies to ETH sent directly to a contract that has no withdrawal functionality.

Fixing Remarks

If a rescue mechanism is desired, it should be introduced carefully.

```
function rescueERC20(
    address token,
    address recipient,
    uint256 amount
) external onlyOwner {
    IERC20(token).transfer(recipient, amount);
}
```

However, because the current contract appears to have relinquished ownership, adding such functionality would require a new deployment or a different governance architecture.

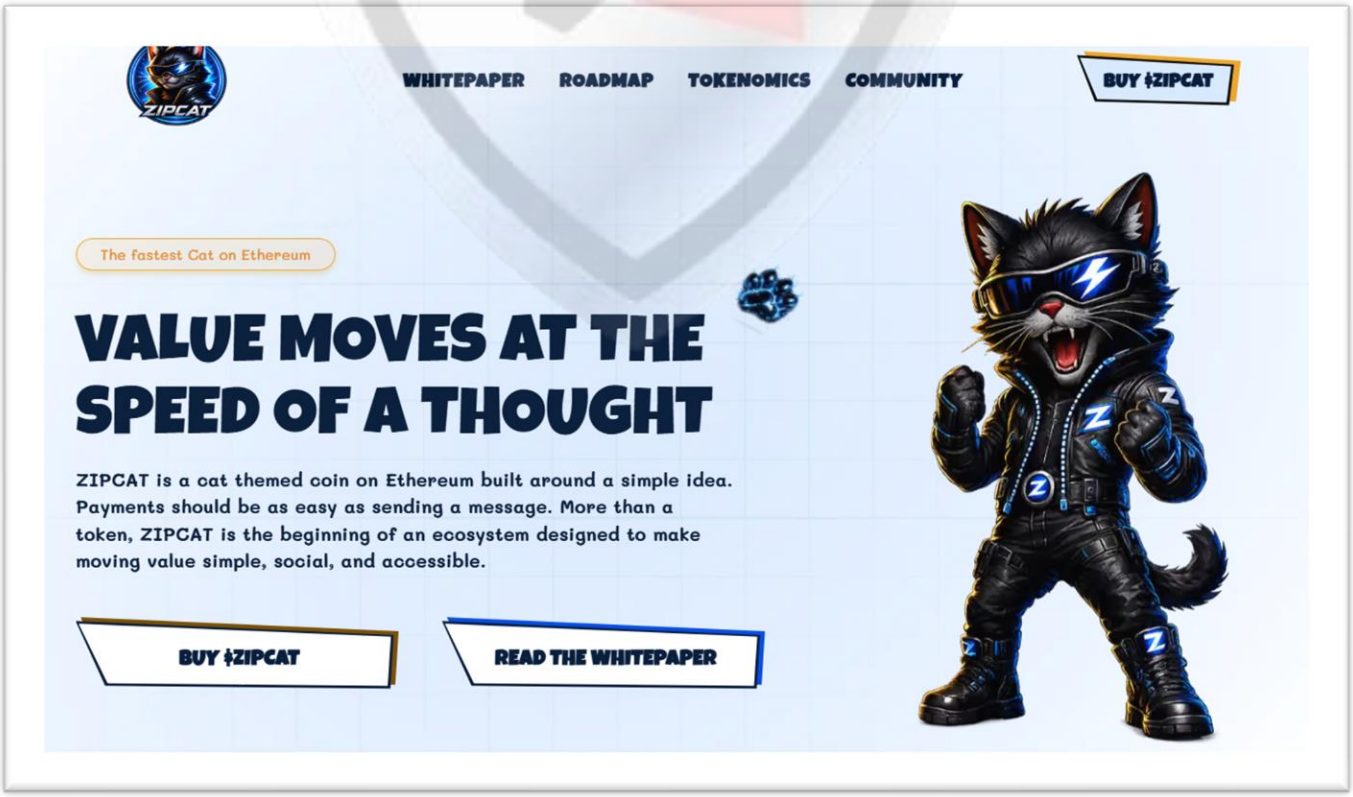
Do not add a rescue function to an existing immutable/renounced contract without carefully considering its trust implications.

MANUAL REVIEW

ZIPCAT is a cat themed coin on Ethereum built around a simple idea. Payments should be as easy as sending a message. More than a token, ZIPCAT is the beginning of an ecosystem designed to make moving value simple, social, and accessible.

TOKEN NAME: ZIP CAT
Ticker: $ZIPCAT
MAX TOTAL SUPPLY: 1,000,000,000,000 ZIPCAT
Chain/Standard: ETHEREUM NETWORK
LAUNGUGE: SOLIDITY

The ZIPCAT Platform Is Launched On the Ethereum Network



Deployer Contract:

0x4C5FC2EFd7F9fBA2c6a9227aBd9d1C3eeFd6bDD7

Audited Files

O.ZIPCAT.sol

Contract Creator Address

0x7d59d1f8BD5B0640a5E324479600A5D925949fDF

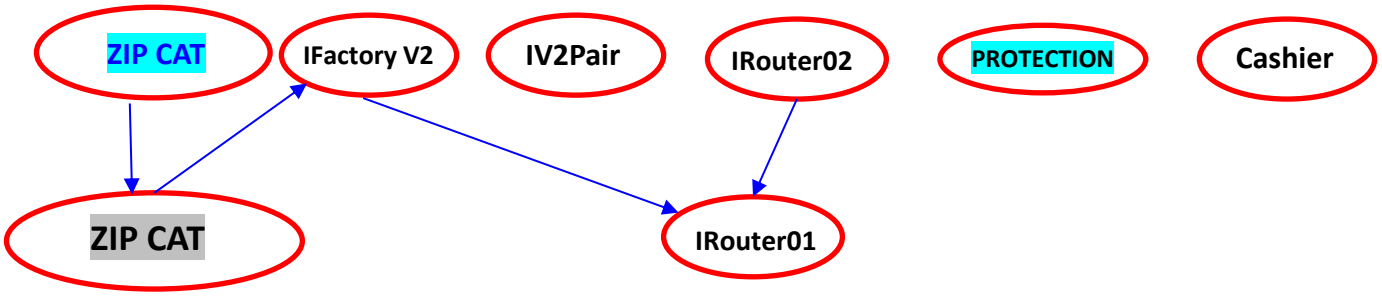
Deployed Contracts:

0x4C5FC2EFd7F9fBA2c6a9227aBd9d1C3eeFd6bDD7

Creator TXH Contracts:

https://etherscan.io/tx/0xd443e01e24390c5be2687ad4f2c862e627ca2da5d75b80f06cadc84eb1be9161

INHERITANCE GRAPH



Identifier	Definition	Severity
CEN-12	Centralization privileges of ZIP CAT	Medium #

Vulnerability 0 : No important security issue detected.
Threat level: Low

```
contract ZipCat is ERC20, Ownable {
    using SafeMath for uint256;

    IUniswapV2Router02 public immutable uniswapV2Router;
    address public immutable uniswapV2Pair;

    bool private swapping;

    address public marketingWallet;
    address public devWallet;

    uint256 public maxTransactionAmount;
    uint256 public swapTokensAtAmount;
    uint256 public maxWallet;

    bool public limitsInEffect = true;
    bool public tradingActive = false;
    bool public swapEnabled = false;
    bool public enableEarlySellTax = true;

    // Anti-bot and anti-whale mappings and variables
    mapping(address => uint256) private _holderLastTransferTimestamp; // to hold
```

VITALBlock security.

ISSUES CHECKING STATUS

	Issue Description	Checking Status
1.	Compiler errors.	PASSED
2.	Race Conditions and reentrancy. Cross-Function Race Conditions.	PASSED
3.	Possible Delay In Data Delivery.	PASSED
4.	Oracle calls.	PASSED
5.	Front Running.	PASSED
6.	Sol Dependency.	PASSED
7.	Integer Overflow And Underflow.	PASSED
8.	DoS with Revert.	PASSED
9.	Dos With Block Gas Limit.	PASSED
10.	Methods execution permissions.	PASSED
11.	Economy Model of the contract.	PASSED
12.	The Impact Of Exchange Rate On the solidity Logic.	PASSED
13.	Private use data leaks.	PASSED
14.	Malicious Event log.	PASSED
15.	Scoping and Declarations.	PASSED
16.	Uninitialized storage pointers.	PASSED
17.	Arithmetic accuracy.	PASSED
18.	Design Logic.	PASSED
19.	Cross-Function race Conditions	PASSED
20.	Save Upon solidity contract Implementation and Usage.	PASSED
21.	Fallback Function Security	PASSED

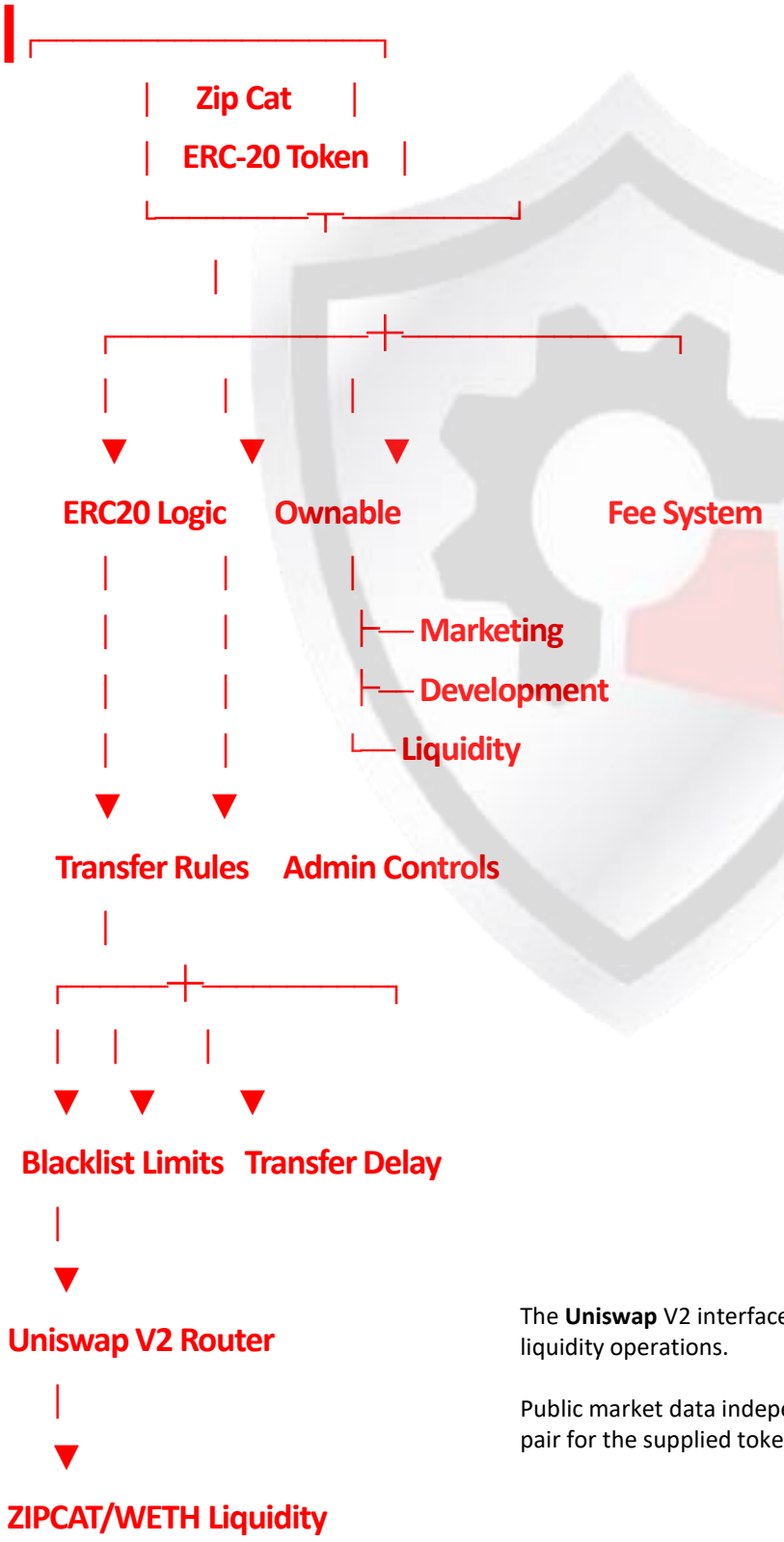
AUDIT RESULT

PASSED

SMART CONTRACT AUDIT OF ZIPCAT

Identifier	Definition	Severity
CEN-02	The visible architecture is:	Minor $

All of the initially minted assets are sent to the contract deployer when deploying the contract. This is Normal for most deployer and/or contract owner .



The **Uniswap** V2 interfaces include router functionality for token swaps and liquidity operations.

Public market data independently identifies a **ZIPCAT/WETH** Uniswap V2 pair for the supplied token address

RECOMMENDATION

Deployer and/or contract owner private keys are secured carefully.

Please refer to PAGE-7 CENTRALIZED PRIVILEGES for a detailed understanding.

ALLEVIATION

The ZIPCAT project team understands the centralization risk. Some functions are provided privileged access to ensure a good runtime behavior in the project

References

1 MITRE. CWE-1041: Use of Redundant Code. https://cwe.mitre.org/data/definitions/1041.html.

2 MITRE. CWE-1099: Inconsistent Naming Conventions for Identifiers. https://cwe.mitre.org/data/definitions/1099.html.

3 MITRE. CWE-561: Dead Code. https://cwe.mitre.org/data/definitions/561.html.

4 MITRE. CWE-563: Assignment to Variable without Use. https://cwe.mitre.org/data/definitions/563.html.

5 MITRE. CWE-663: Use of a Non-reentrant Function in a Concurrent Context. https://cwe.mitre.org/data/definitions/663.html.

6 MITRE. CWE-837: Improper Enforcement of a Single, Unique Action. https://cwe.mitre.org/data/definitions/837.html.

7 MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. https://cwe.mitre.org/data/definitions/841.html.

8 MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

9 MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

10 MITRE. CWE CATEGORY: Concurrency. https://cwe.mitre.org/data/definitions/557.html.

11 MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

12 OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

Identifier	Definition	Severity
COD-10	Third Party Dependencies	Minor

Smart contract is interacting with third party protocols e.g., Pancakeswap router, cashier contract, protections contract. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and exploited. Moreover, upgrades in third parties can create severe impacts, e.g., increased transactional fees, deprecation of previous routers, etc.

RECOMMENDATION

Inspect and validate third party dependencies regularly, and mitigate severe impacts whenever necessary.

DISCLAIMERS

Vital Block Security provides the easy-to-understand audit of Solidity, Move and Raw source codes (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities, and centralization exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analyzed, nor do they provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies, they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without InterFi Network's prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way

to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort.

FOR AVOIDANCE OF DOUBT, SERVICES, INCLUDING ANY ASSOCIATED AUDIT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

TECHNICAL DISCLAIMER

ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, VITAL BLOCK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO SERVICES, AUDIT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, VITAL BLOCK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM THE COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

WITHOUT LIMITING THE FOREGOING, VITAL BLOCK MAKES NO WARRANTY OF ANY KIND THAT ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET THE CLIENT'S OR ANY OTHER INDIVIDUAL'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. Vital Block does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.

LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than Vital Block. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites and social accounts owners. You agree that Vital block Security is not responsible for the content or operation of such websites and social accounts and that Vital Block shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.

CERTIFICATE BY VITAL BLOCK SECURITY

VITALBlock SKYNET

ABOUT VITAL BLOCK

Vital Block provides intelligent blockchain Security Solutions. We provide solidity and Raw Code Review, testing, and auditing services. We have Partnered with 15+ Crypto Launchpads, audited 1850+ smart contracts, and analyzed Over 200,000+ code lines. We have worked on major public blockchains e.g., Ethereum, Binance, Cronos, Doge, Polygon, Avalanche, Metis, Fantom, Bitcoin Cash, Aptos, Oasis, etc.

Vital Block is Dedicated to Making Defi & Web3 A Safer Place. We are Powered by Security engineers, developers, UI experts, and blockchain enthusiasts. Our team currently consists of 5 core members, and 8+ casual contributors.

1000+
Audits Completed

$12B+
Secured

1M+
Lines of Code Audited

@vb_audit

@Vitalblock

@vitalblock

github.com/VBS-LABS

Info@vitalblock.org

www.vitalblock.org

Blockchain Security | Smart Contract Audit | KYC Certification | SAFU .

MADE IN CANADA